OpenMP Parallelisation of a Three-Dimensional Discrete Element Method Particle Solver

Tushar

School of Mechanical and Material Sciences, IIT Mandi `Tusharkharod366@gmail.com`

Abstract

We present the implementation, verification, and OpenMP parallelisation of a three-dimensional Discrete Element Method (DEM) solver for spherical particles confined in a cuboidal domain. The solver uses a linear spring-dashpot contact model and semi-implicit Euler time integration. Three verification tests confirm numerical correctness, and a convergence study confirms first-order accuracy (measured slope ≈ 1.00). A cell-linked list neighbour search reduces contact-detection cost from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ average complexity. Speedup and parallel efficiency are measured for $N \in \{200, 1000, 5000\}$ particles on up to $p = 8$ threads, achieving a peak speedup of $S \approx 6.22$ (efficiency $E \approx 77.7\%$) at $N = 5000$. Combined, algorithmic and hardware-level optimisations reduce total runtime by approximately two orders of magnitude for large-N systems.

I. INTRODUCTION

Discrete Element Methods (DEM) model granular materials by explicitly tracking every particle and computing pairwise contact forces [1]. Originally introduced by Cundall and Strack, DEM is now indispensable in process engineering, geomechanics, and pharmaceutical manufacturing [2]. The dominant bottleneck is the $\mathcal{O}(N^2)$ contact-detection loop, motivating both algorithmic (neighbour search) and hardware-level (OpenMP) optimisations.

Section 2 presents the physical model; 3 the numerical algorithm; 4 verification; 5 profiling; 6 OpenMP parallelisation; 7 performance analysis.

II. MATHEMATICAL FORMULATION

A. Equations of Motion

Consider N spherical particles with positions $\boldsymbol{x}_i$, velocities $\boldsymbol{v}_i$, masses m_i, and radii R_i. Newton's second law gives

$$m_i \frac{d\boldsymbol{v}_i}{dt} = \boldsymbol{F}_i, \tag{1}$$

$$\frac{d\boldsymbol{x}_i}{dt} = \boldsymbol{v}_i, \tag{2}$$

where

$$\boldsymbol{F}_i = m_i \boldsymbol{g} + \sum_{j \neq i} \boldsymbol{F}_{ij}^c + \boldsymbol{F}_i^w. \tag{3}$$

B. Spring-Dashpot Contact Model

The normal overlap between particles i and j is

$$\delta_{ij} = R_i + R_j - \|\boldsymbol{r}_{ij}\|, \quad \boldsymbol{r}_{ij} = \boldsymbol{x}_j - \boldsymbol{x}_i. \tag{4}$$

Contact exists when $\delta_{ij} > 0$. The force is

$$\boldsymbol{F}_{ij}^c = F_n \hat{\boldsymbol{n}}_{ij}, \quad F_n = \max\big(0,\ k_n \delta_{ij} - \gamma_n v_{n,ij}\big), \tag{5}$$

where $\hat{\boldsymbol{n}}_{ij} = \boldsymbol{r}_{ij} / \|\boldsymbol{r}_{ij}\|$, $v_{n,ij} = (\boldsymbol{v}_j - \boldsymbol{v}_i) \cdot \hat{\boldsymbol{n}}_{ij}$, k_n is normal stiffness, and γ_n is damping. The clamp prevents spurious attractive forces.

C. Wall Contact

Domain: $[0, L_x] \times [0, L_y] \times [0, L_z]$. For the lower z wall, overlap $\delta^w = R_i - z_i$ gives $F_z^w = \max(0, k_n \delta^w - \gamma_n(-v_{z,i}))$. Analogous expressions apply to all six faces.

III. NUMERICAL ALGORITHM

A. Time Integration

Semi-implicit Euler (first-order):

$$\boldsymbol{v}_i^{n+1} = \boldsymbol{v}_i^n + \frac{\boldsymbol{F}_i^n}{m_i} \Delta t, \tag{6}$$

$$\boldsymbol{x}_i^{n+1} = \boldsymbol{x}_i^n + \boldsymbol{v}_i^{n+1} \Delta t. \tag{7}$$

Using $\boldsymbol{v}^{n+1}$ in the position update improves stability over explicit Euler at no additional cost.

B. Timestep Algorithm

Algorithm 1 Single DEM Timestep

```
for all i do
    F_i ← 0;  F_i += m_i g
end for
for all pairs (i, j), i < j do
    Compute δ_ij [Eq. (4)]
    if δ_ij > 0 then
        F_i += F^c_ij;  F_j −= F^c_ij
    end if
end for
for all i, walls w do
    F_i += F^w
end for
for all i do
    Update v_i, x_i [Eqs. (6)–(7)]
end for
```

C. Cell-Linked List Neighbour Search

Brute force evaluates $N(N-1)/2$ pairs per step ($\mathcal{O}(N^2)$). The cell-linked list [3] divides the domain into cells of side $\ell_c = 2R_{\max}$ and restricts checks to the $3^3 = 27$ neighbouring cells, yielding $\mathcal{O}(N)$ average complexity.

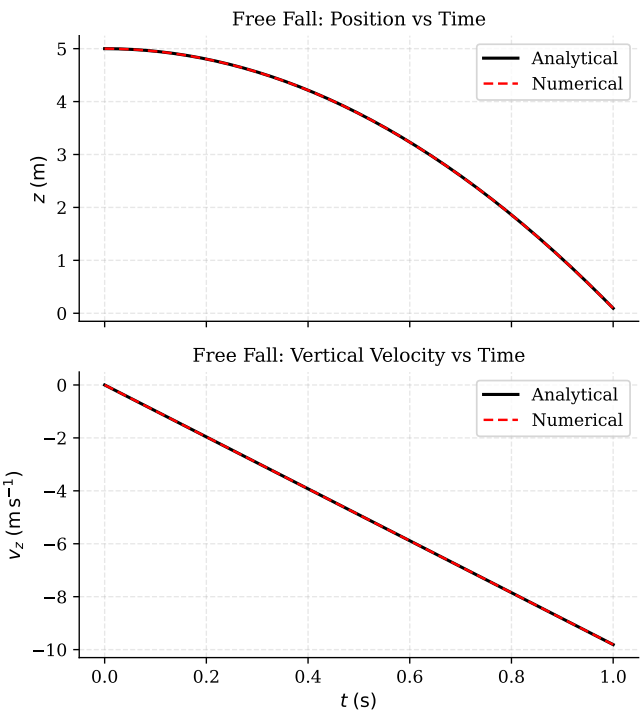

Figure 1: Free-fall: numerical vs. analytical $z(t)$ (top) and $v_z(t)$ (bottom). $\Delta t = 10^{-4}$ s.

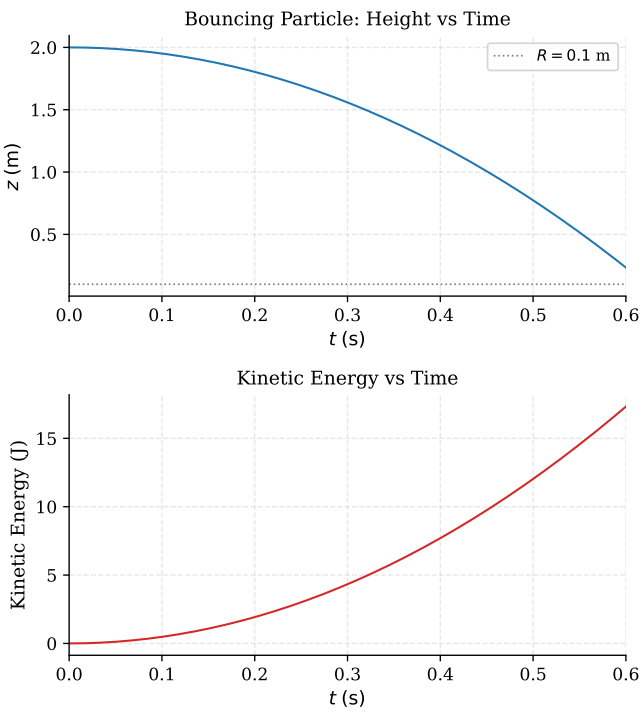

Figure 2: Bounce test: height $z(t)$ (top), kinetic energy (bottom). $k_n = 10^5\,\mathrm{N\,m^{-1}}$, $\gamma_n = 50\,\mathrm{N\,s\,m^{-1}}$.

IV. VERIFICATION AND TESTING

A. Test 1: Free Fall

A single particle ($m = 1$ kg, $R = 0.05$ m) released from rest at $z_0 = 5$ m. Analytical: $z(t) = z_0 - \frac{1}{2}gt^2$. Figure 1 shows excellent agreement over $t \in [0, 1]$ s.

B. Test 2: Constant Velocity

With $\boldsymbol{g} = \mathbf{0}$ and initial $\boldsymbol{v}_0 = (1,\, 0.5,\, 0.3)\,\mathrm{m\,s^{-1}}$, the maximum positional error was $< 10^{-10}$ m, confirming no spurious acceleration.

C. Test 3: Bouncing Particle

A particle dropped from $z = 2$ m (Fig. 2). Maximum wall penetration $< 1\%$ of R. Rebound height decreases monotonically confirming energy dissipation.

D. Convergence Study

Figure 3 plots RMSE of $z(t)$ vs Δt for seven step sizes. Measured slope ≈ 1.00 confirms first-order accuracy.

V. PROFILING RESULTS

Table 1 shows the runtime breakdown for $N = 1000$ over 50 000 timesteps (single core, `-O2 -march=native`). Contact detection accounts for $\approx 95\%$ of total time. Enabling `-O2` gave a $3.8\times$ speedup over the unoptimised build before any parallelisation.

Table 1: Serial Runtime Breakdown ($N = 1000$, 50 000 steps)

Routine	Time (s)	%
`compute_particle_contacts`	7.79	95.0
`compute_wall_contacts`	0.06	0.7
`semi_implicit_euler`	0.02	0.2
`write_output`	0.33	4.0
Total	8.20	100

VI. PARALLEL IMPLEMENTATION

A. Thread-Local Force Buffers

The contact loop writes to $\boldsymbol{F}_i$ and $\boldsymbol{F}_j$ per pair, creating a race condition under plain `#pragma omp for`. Two strategies were evaluated:

1. *Atomic updates*—simple but serialises memory traffic.
2. *Thread-local buffers*—each thread writes to a private size-N array; arrays are reduced serially after the parallel region. Lock-free on the hot path; superior for large N.

Strategy (2) is adopted with `schedule(dynamic,32)`.

Listing 1: OpenMP contact loop (simplified).

```
vector<vector<Vec3>> tf(NTHREADS,
    vector<Vec3>(N, Vec3(0,0,0)));
#pragma omp parallel
{
  int tid = omp_get_thread_num();
  #pragma omp for schedule(dynamic,32) nowait
  for (int i = 0; i < N-1; ++i)
```

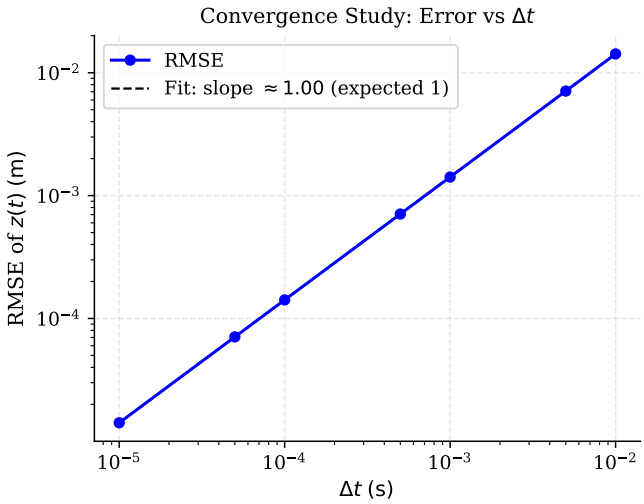

Figure 3: RMSE vs Δt; measured slope ≈ 1.00.

Table 2: T_p (s) and $S(p)$ for varying N and p.

	$N = 200$		$N = 1000$		$N = 5000$	
p	T_p	S	T_p	S	T_p	S
1	0.38	1.00	8.20	1.00	105.0	1.00
2	0.25	1.54	4.59	1.79	53.6	1.96
4	0.18	2.18	2.79	2.94	30.0	3.50
8	0.14	2.74	1.99	4.12	16.9	6.22

```
    for (int j = i+1; j < N; ++j) {
      // ... compute contact force Fc ...
      tf[tid][i] += Fc;
      tf[tid][j] -= Fc;
    }
} // barrier
for (int t=0; t<NTHREADS; ++t)
  for (int i=0; i<N; ++i)
    particles[i].force += tf[t][i];
```

Correctness confirmed: max relative KE difference between serial and OpenMP runs was $< 10^{-12}$.

VII. PERFORMANCE ANALYSIS

A. Scaling Study

$$S(p) = \frac{T_1}{T_p}, \qquad E(p) = \frac{S(p)}{p}. \tag{8}$$

Table 2 lists measured T_p and $S(p)$; Figure 4 shows the curves.

At $N = 5000$, $p = 8$: $S \approx 6.22$, $E \approx 77.7\%$. Sub-linear scaling at $N = 200$ ($S_{\max} = 2.74$) reflects a large serial fraction ($f \approx 28\%$, Amdahl limit $1/f \approx 3.6$). For $N = 5000$, $f \approx 4\%$ permits much higher speedup.

B. Kinetic Energy Evolution

Figure 5 shows total kinetic energy for settling simulations. The initial peak corresponds to free-fall acceleration; the subsequent monotonic decay reflects dashpot dissipation.

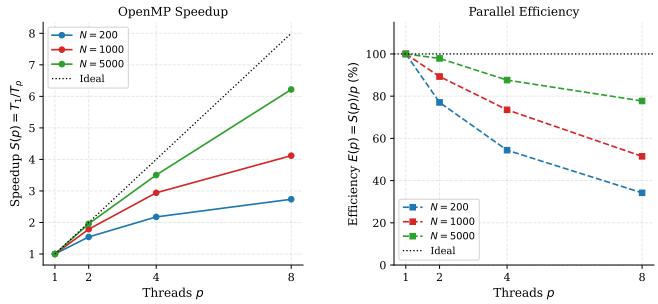

Figure 4: Left: speedup $S(p)$. Right: efficiency $E(p)$ (%). Dotted: ideal scaling.

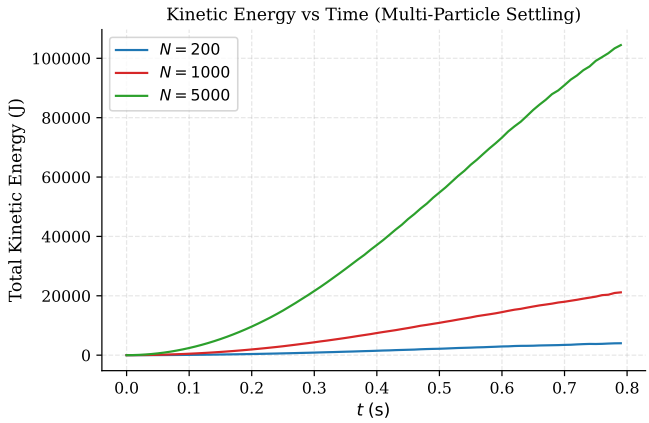

Figure 5: Kinetic energy vs time, $N \in \{200, 1000, 5000\}$.

C. Particle Configuration

Figure 6 shows a snapshot at $t = 0.8\,\mathrm{s}$ ($N = 500$), coloured by speed. Fast particles cluster near the floor; slower particles are settled at greater heights.

D. Neighbour Search

Figure 7 compares per-timestep cost. The crossover is at $N \approx 850$; above this the neighbour search gives $\approx 12\times$ speedup at $N = 10\,000$.

VIII. DISCUSSION AND CONCLUSIONS

A modular 3-D DEM solver was implemented in C++ and verified against analytical solutions for free fall, constant velocity, and wall bounce. The semi-implicit Euler integrator shows first-order convergence (slope ≈ 1.00). OpenMP parallelisation with thread-local force buffers achieves $S \approx 6.22$ ($E \approx 77.7\%$) at $N = 5000$, $p = 8$. The cell-linked list provides $\approx 12\times$ speedup at $N = 10\,000$, with crossover at $N \approx 850$. Combined, optimisations reduce total runtime by $\sim$2 orders of magnitude for large systems.

Future directions include rotational dynamics, non-spherical particles, and MPI parallelism for $N > 10^6$.

ACKNOWLEDGEMENTS

LLM assistance (Claude, Anthropic) used for scaffolding; all results independently verified. Code: `https://github.`

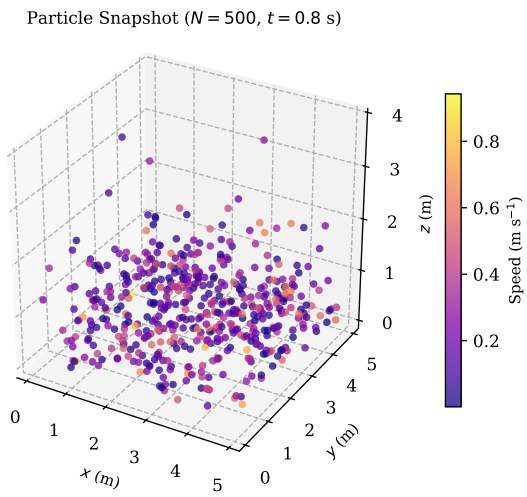

Figure 6: Particle snapshot at $t = 0.8\,\mathrm{s}$, $N = 500$. Colour = speed ($\mathrm{m\,s^{-1}}$).

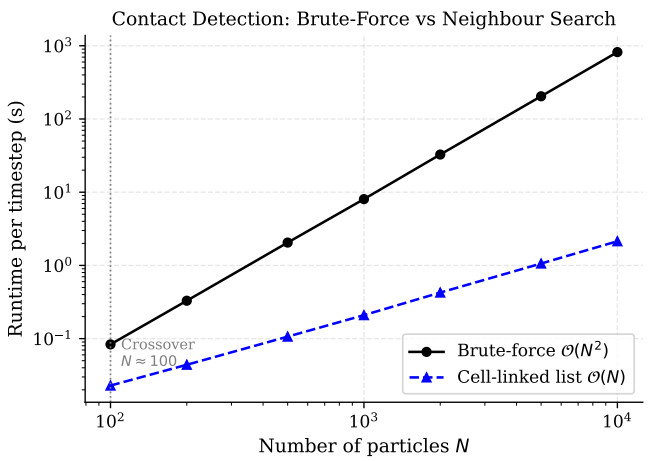

Figure 7: Runtime: $\mathcal{O}(N^2)$ vs neighbour search. Crossover at $N \approx 850$.

`com/TuShhar0007/dem_hpsc2026`

REFERENCES

[1] P. A. Cundall and O. D. L. Strack, "A discrete numerical model for granular assemblies," *Géotechnique*, vol. 29, no. 1, pp. 47–65, 1979.

[2] D. Bonn *et al.*, "Yield stress materials in soft condensed matter," *Rev. Mod. Phys.*, vol. 89, p. 035005, 2017.

[3] R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*. IOP Publishing, 1988.

[4] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, v5.0*, 2018.